EXPRESS example model Car Registration Authority

Introduction

This document contains a complete and documented EXPRESS model for the Car Registration Authority example; an EXPRESS-G version of the model is also included.

Scope

The model has to do with the registration of cars and is limited to the scope of interest of the Registration Authority. This Authority exists for the purpose of:

- Knowing who is or was the registered owner of a car at any time from construction to destruction of the car;
- To monitor laws regarding the transfer of ownership of cars;
- To monitor laws regarding the fuel consumption of cars;
- To monitor laws regarding manufacturers of cars.

Model overview

The model is described using both EXPRESS and EXPRESS-G. The EXPRESS definitions are primary and the EXPRESS-G diagrams are to assist in understanding the primary model. If there is any conflict between the EXPRESS and EXPRESS-G, then the EXPRESS takes precedence.

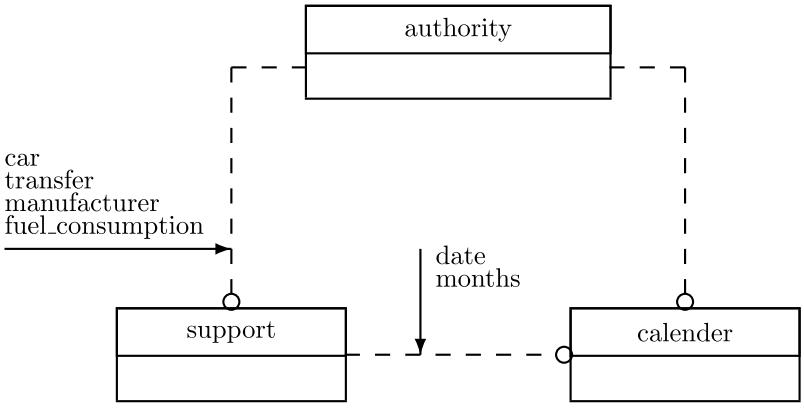


Figure 1. Complete schema-level model for Registration Authority example

The model consists of three schemas, as shown in Figure Complete schema-level model for Registration Authority example. The schema `tt "authority"` is the primary schema. It references items from the two ancilliary schemas, namely `tt "support"` and `tt "calendar"`. The `tt "support"` schema also references items from the `tt "calendar"` schema.

Authority schema

This schema is the primary one in the model and is principally concerned with the main functions of the Registration Authority.

The schema imports definitions from two sources, namely the `tt "support"` and the `tt "calendar"` schemas.

Figure Complete entity-level model of the Authority schema. is an EXPRESS-G complete entity-level model for this schema.

EXPRESS specification:

```
*)
SCHEMA authority;
  REFERENCE FROM support (car,
        transfer,
        manufacturer,
        fuel_consumption,
        mnfg_average_consumption);
  REFERENCE FROM calendar (current_date);
(*
```

Entity definitions

history

A `tt "history"` records the transfers of ownership of a `tt "car"` over its lifetime. A `tt "history"` must be kept for a certain period after the `tt "car"` is destroyed, after which the ownership records may be destroyed.

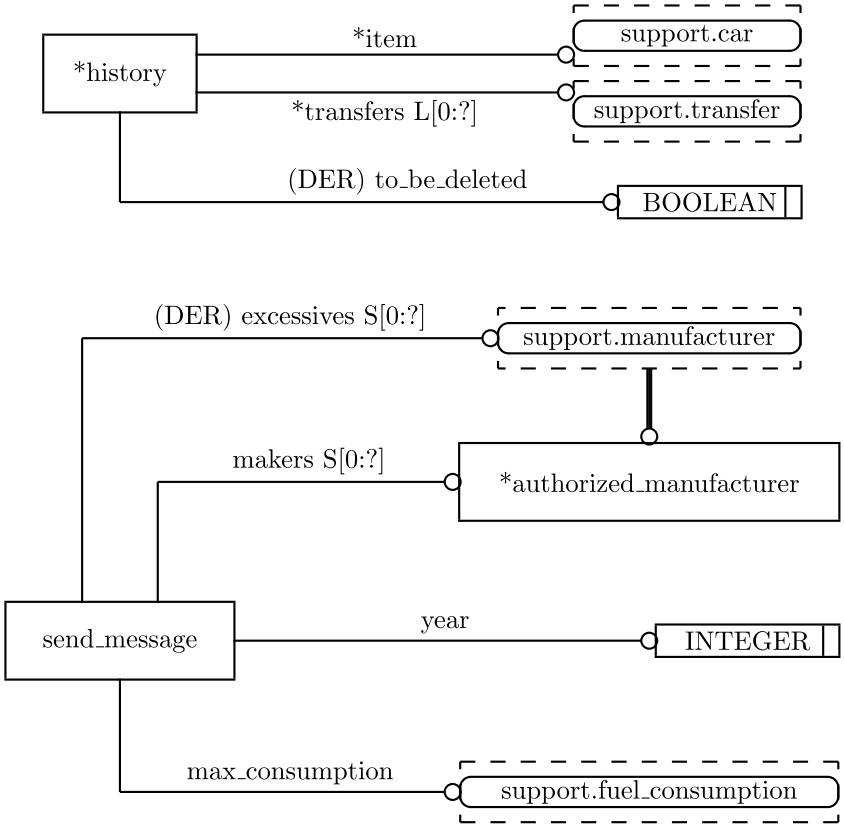


Figure 2. Complete entity-level model of the Authority schema.

EXPRESS specification:

```
*)
ENTITY history;
  item : car;
  transfers : LIST [0:?] OF UNIQUE transfer;
DERIVE
  to_be_deleted : BOOLEAN := too_old(SELF);
UNIQUE
  un1 : item;
WHERE
  one_car  : single_car(SELF);
  ordering : exchange_ok(transfers);
END_ENTITY;
(*
```

Attribute definitions

item:

The `tt "car"` whose ownership history is being tracked.

transfers:

The ownership `tt "transfer"` records of the `tt "item"`.

to_be_deleted:

A flag which indicates that this `tt "history"` record may be deleted because the `tt "item"` has

been destroyed (TRUE), or that the record shall not be deleted (FALSE).

Propositions

un1:

The value of `tt "item"` shall be unique across all instances of `tt "history"`.

one_car:

Each `tt "transfer"` collected in a `tt "history"` shall be of the same `tt "car"`.

ordering:

The list of `tt "transfer"` shall be in increasing historical order.

authorized_manufacturer

An `tt "authorized manufacturer"` is a `tt "manufacturer"` who has been given permission by the Registration Authority to make cars.

EXPRESS specification:

```
*)
ENTITY authorized_manufacturer
  SUBTYPE OF (manufacturer);
END_ENTITY;
(*
```

send_message

In January each year the Registration Authority shall send a message to each `tt "manufacturer"` whose cars' average fuel consumption exceeds a certain limit, which may vary from year to year.

EXPRESS specification:

```
*)
ENTITY send_message;
  max_consumption : fuel_consumption;
  year            : INTEGER;
  makers          : SET [0:?] OF authorized_manufacturer;
DERIVE
  excessives : SET [0:?] OF manufacturer := guzzlers(SELF);
END_ENTITY;
(*
```

Attribute definitions

max_consumption:

The legal maximum average fuel consumption.

year:

The year for which the `tt "max consumption"` value applies.

makers:

The `tt "authorized manufacturers"` operating during the `tt "year"`.

excessives:

The `tt "manufacturers"` whose cars exceed the consumption limit.

Rule definitions

max_number

No more than five `tt "authorized manufacturers"` are permitted at any one time.

EXPRESS specification:

```
*)
RULE max_number FOR (authorized_manufacturer);
WHERE
  max_of_5 : SIZEOF(authorized_manufacturer) <= 5;
END_RULE;
(*
```

Propositions

max_of_5:

The rule is violated if there are more than five `tt "authorized manufacturers"` at any time.

Function and procedure definitions

guzzlers

This function returns the set of `tt "manufacturers"` whose cars exceed an average fuel consumption limit.

Parameters

par:

An instance of a `tt "send message"` entity.

RESULT:

A set of instances of `tt "manufacturer"` whose cars' average fuel consumption is excessive.

EXPRESS specification:

```
*)
FUNCTION guzzlers(par : send_message) : SET OF manufacturer;
LOCAL
  result : SET OF manufacturer := [];
  mnfs   : SET OF manufacturer := par.makers;
  limit  : fuel_consumption := par.max_consumption;
  time   : INTEGER := par.year;
END_LOCAL;
  REPEAT i := 1 TO SIZEOF(mnfs);
    IF (mnfg_average_consumption(mnfs[i],time) > limit) THEN
      result := result + mnfs[i];
    END_IF;
  END_REPEAT;
RETURN(result);
END_FUNCTION;
(*
```

too_old

This function calculates whether the `tt "car"` in a `tt "history"` was destroyed more than two years ago.

Parameters

par:

An instance of a `tt "history"`.

RESULT:

A Boolean value. TRUE if the `tt "car"` in the input `tt "history"` was destroyed two or more years ago; otherwise FALSE.

EXPRESS specification:

```
*)
FUNCTION too_old(par : history) : BOOLEAN;
  (* The function returns TRUE if the input history is
    outdated. That is, if it is of an item that was destroyed
    more than 2 years ago. *)
  IF ('SUPPORT.DESTROYED_CAR' IN par.item) THEN
    IF (current_date.year-par.item.destroyed_on.year >= 2) THEN
      RETURN(TRUE);
    END_IF;
  END_IF;
  RETURN(FALSE);
END_FUNCTION;
(*
```

exchange_ok

This function checks whether or not the tt "transfers" in a list are ordered.

Parameters

par

A list of tt "transfer" instances.

RESULT

A Boolean value. TRUE if the recipient in the Nth transfer is the same as the giver in the (N+1)^{th} transfer.

EXPRESS specification:

```
*)
FUNCTION exchange_ok(par : LIST OF transfer) : BOOLEAN;
  (* returns TRUE if the "to owner" in the N'th transfer of a
    car is the "from owner" in the N+1'th transfer *)
  REPEAT i := 1 TO (SIZEOF(par) - 1);
    IF (par[i].new :<>: par[i+1].prior) THEN
      RETURN (FALSE);
    END_IF;
  END_REPEAT;
  RETURN (TRUE);
END_FUNCTION;
(*
```

single_car

This function checks whether or not the tt "car" in a transfer tt "history" is the same tt "car" specified in each individual tt "transfer".

Parameters

par:

A tt "history" instance.

RESULT:

A Boolean value. TRUE if the tt "history" and all its tt "transfers" are of the same tt "car", otherwise FALSE.

EXPRESS specification:

```
*)
FUNCTION single_car(par : history) : BOOLEAN;
  (* returns TRUE if a history is of a single car *)
  REPEAT i := 1 TO SIZEOF(par.transfers);
```

```
    IF (par.item :<>: par.transfers[i].item) THEN
      RETURN (FALSE);
    END_IF;
  END_REPEAT;
  RETURN (TRUE);
END_FUNCTION;
(*
```

Entity classification structure

The following indented listing shows the entity classification structure. Entities in upper case characters are defined in this schema. Entities in lower case characters are defined in other schemas.

```
HISTORY
manufacturer (in schema support)
  AUTHORIZED_MANUFACTURER
SEND_MESSAGE
```

```
*)
END_SCHEMA;  -- end of authority schema
(*
```

Support schema

This schema contains supporting definitions for the primary `tt "authority"` schema.

An EXPRESS-G model of the contents of this schema is given in Figure Complete entity-level model of the Support schema. and in Figure Complete entity-level model of the Support schema..

The schema imports definitions from the `tt "calendar"` schema.

EXPRESS specification:

```
*)
SCHEMA support;
  REFERENCE FROM calendar (date, months, days_between);
(*
```

Type definitions

name

The ‘name’ of something. A human interpretable name which may identify some object, thing or

person, etc. For example, `tt "Widget Company, Inc."`.

EXPRESS specification:

```
*)
TYPE name = STRING;
END_TYPE;
(*
```

identification_no

A character string which may be used as the 'identification number' for a particular instance of some object. This is typically a mixture of alphanumeric characters and other symbols. For example, `tt "D20-736597WP23"`.

EXPRESS specification:

```
*)
TYPE identification_no = STRING;
END_TYPE;
(*
```

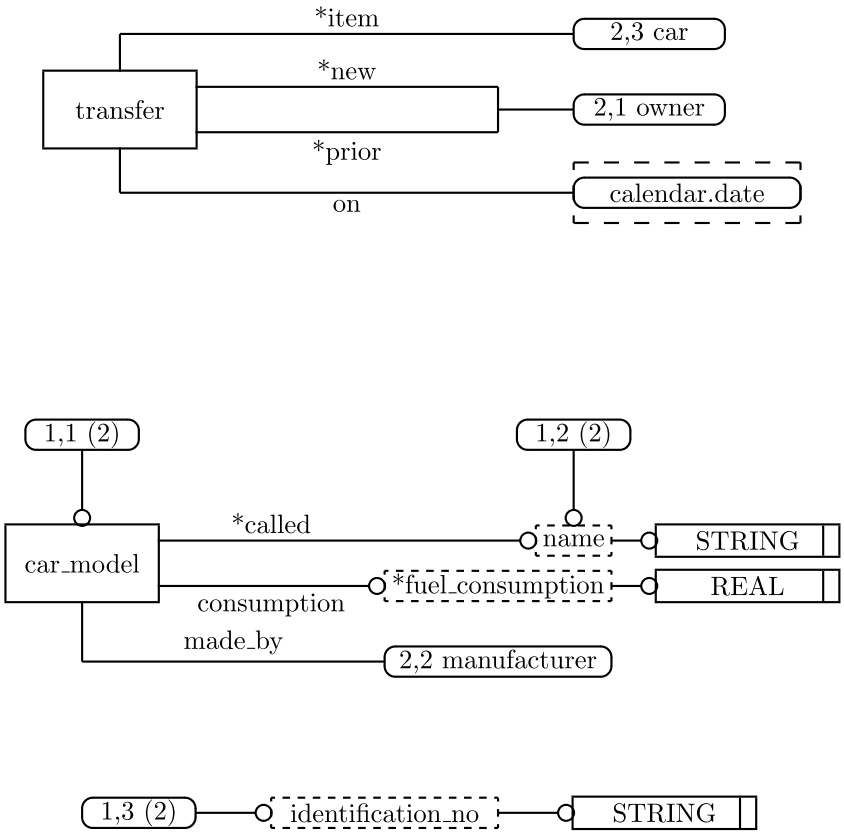


Figure 3. Complete entity-level model of the Support schema.

fuel_consumption

A measure of the fuel consumption of some powered device.

EXPRESS specification:

```
*)
TYPE fuel_consumption = REAL;
WHERE
  range : {4.0 <= SELF <= 25.0};
END_TYPE;
(*
```

Propositions

range:

The value is limited to lie in the range 4 to 25 inclusive.

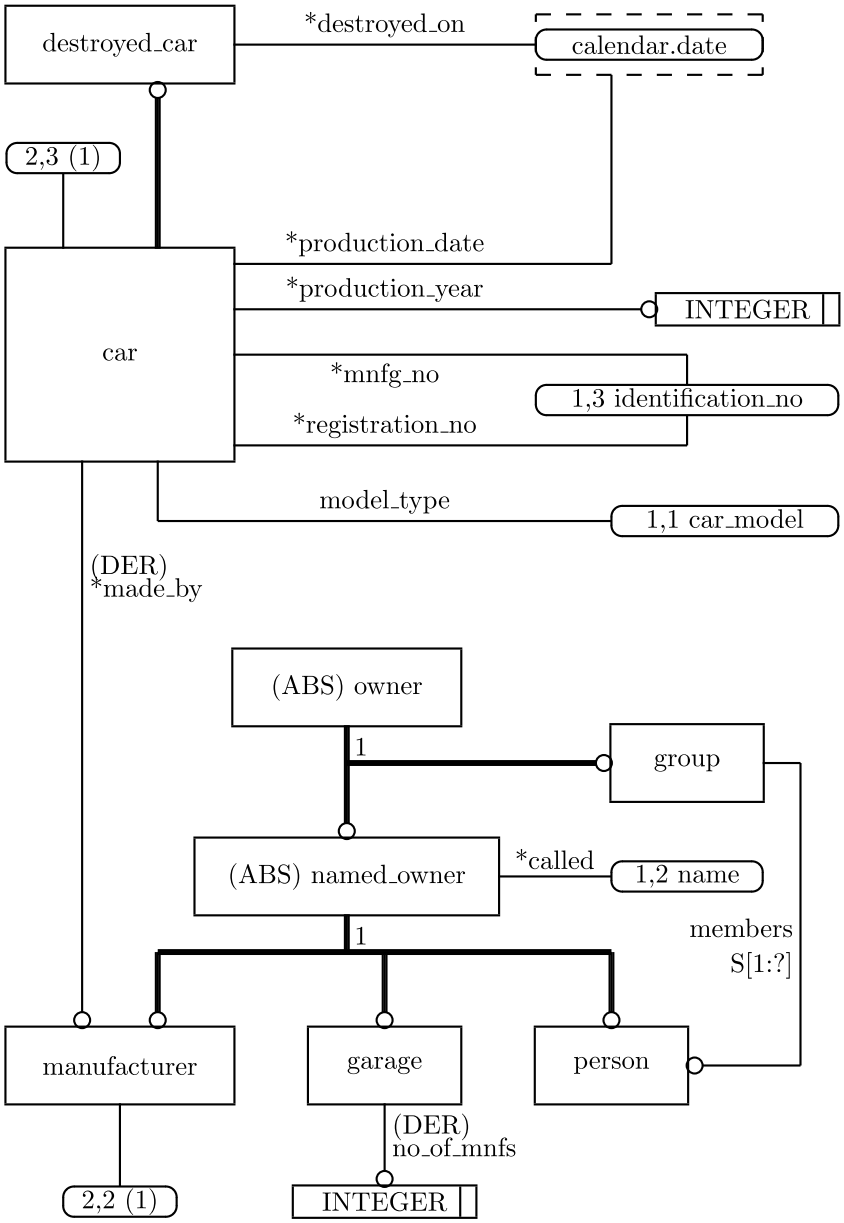


Figure 4. Complete entity-level model of the Support schema.

Entity definitions

transfer

A record of a transfer of a `tt "car"` from one owner to a new owner.

EXPRESS specification:

```
*)
ENTITY transfer;
  item  : car;
  prior : owner;
  new   : owner;
  on    : date;
WHERE
  wr1 : NOT ('SUPPORT.MANUFACTURER' IN TYPEOF(new));
  wr2 : (NOT ('SUPPORT.MANUFACTURER' IN TYPEOF(prior))) XOR
     (('SUPPORT.MANUFACTURER' IN TYPEOF(prior)) AND
     ('SUPPORT.GARAGE' IN TYPEOF (new)));
  wr3 : (NOT ('SUPPORT.GARAGE' IN TYPEOF(prior))) XOR
     (('SUPPORT.GARAGE' IN TYPEOF(prior)) AND
     (('SUPPORT.PERSON' IN TYPEOF(new)) XOR
     ('SUPPORT.GROUP' IN TYPEOF(new))));
  wr4 : (NOT ('SUPPORT.DESTROYED_CAR' IN TYPEOF(item)) XOR
     (('SUPPORT.DESTROYED_CAR' IN TYPEOF(item)) AND
     (days_between(on, item\destroyed_car.destroyed_on) > 0)));
END_ENTITY;
(*
```

Attribute definitions

item:

The `tt "car"` being transferred.

prior:

The prior owner of the `tt "item"`.

new:

The new owner of the `tt "item"`.

on:

The `tt "date"` of the `tt "transfer"`.

Propositions

wr1:

A `tt "car"` cannot be transferred to a `tt "manufacturer"`.

wr2:

A `tt "manufacturer"` can only transfer a `tt "car"` to a `tt "garage"`.

wr3:

A `tt "garage"` can only transfer a `tt "car"` to either a `tt "person"` of a `tt "group"` of people.

wr4:

A `tt "car"` which has been destroyed cannot be transferred.

car

A `tt "car"`.

EXPRESS specification:

```
*)
ENTITY car;
  model_type      : car_model;
  mnfg_no         : identification_no;
  registration_no : identification_no;
  production_date : date;
  production_year : INTEGER;
DERIVE
  made_by : manufacturer := model_type.made_by;
UNIQUE
  joint  : made_by, mnfg_no;
  single : registration_no;
WHERE
  jan_prod : (production_year = production_date.year) XOR
             ((production_date.month = months.January) AND
              (production_year = production_date.year - 1));
END_ENTITY;
(*
```

Attribute definitions

model_type:

The `tt "car model"`.

mnfg_no:

An identification number of the `tt "car"` assigned by the car's manufacturer.

registration_no:

An identification number for the `tt "car"` assigned by the Registration Authority.

production_date:

The date on which the car was produced.

production_year:

The registered year of production of the `tt "car"`.

made_by:

The `tt "manufacturer"` of the `tt "car"`.

Propositions

joint:

The `tt "mnfg no"` given to a `tt "car"` is unique for the given car manufacturer.

single:

Each car is given a unique `tt "registration no"` by the Registration Authority.

jan_prod:

The registered `tt "production year"` is the same as the year in which the car was produced, except that cars produced in January may be registered as having been produced in the previous year.

destroyed_car

A `tt "car"` may be destroyed, in which case its date of destruction is recorded.

EXPRESS specification:

```
*)
ENTITY destroyed_car
  SUBTYPE OF (car);
  destroyed_on : date;
WHERE
  dates_ok : days_between(production_date, destroyed_on) >= 0;
END_ENTITY;
(*
```

Attribute definitions

destroyed_on:

The date on which the `tt "car"` was destroyed.

Propositions

dates_ok:

A `tt "car"` cannot be destroyed before it has been made.

car_model

A particular type of `tt "car"`.

EXPRESS specification:

```
*)
ENTITY car_model;
  called      : name;
  made_by     : manufacturer;
  consumption : fuel_consumption;
UNIQUE
  un1 : called;
END_ENTITY;
(*
```

Attribute definitions

called:

The name of the model.

made_by:

The `tt "manufacturer"` of the model.

consumption:

The average fuel consumption of all cars of this model type.

Propositions

un1:

Each `tt "car model"` has a distinct name.

owner

An owner of a `tt "car"`. Owners are categorized into `tt "named owner"` and `tt "group"`.

EXPRESS specification:

```
*)
ENTITY owner
  ABSTRACT SUPERTYPE OF (ONEOF(named_owner,
                               group));
END_ENTITY;
(*
```

named_owner

An `tt "owner"` who has a name. These are categorized into `tt "manufacturer"`, `tt "garage"` and `tt "person"`.

EXPRESS specification:

```
*)
ENTITY named_owner
  ABSTRACT SUPERTYPE OF (ONEOF(manufacturer,
                               garage,
                               person))
  SUBTYPE OF (owner);
  called : name;
UNIQUE
  un1 : called;
END_ENTITY;
(*
```

Attribute definitions

called:

The name of the `tt "owner"`.

Propositions

un1:

Owner's names are unique.

manufacturer

A type of named car owner. Manufacturers may also manufacture cars.

EXPRESS specification:

```
*)
ENTITY manufacturer
  SUBTYPE OF (named_owner);
END_ENTITY;
(*
```

garage

A type of named car owner.

EXPRESS specification:

```
*)
ENTITY garage
  SUBTYPE OF (named_owner);
DERIVE
  no_of_mnfs : INTEGER := dealer_for_mnfs(SELF);
WHERE
  wr1 : {1 <= no_of_mnfs <= 3};
```

```
END_ENTITY;
(*
```

Attribute definitions

no_of_mnfs:

The number of different manufacturers of the cars owned by the `tt "garage"`.

Propositions

wr1:

At any particular time, a `tt "garage"` shall not own cars made by more than three manufacturers.

person

A type of named car owner.

EXPRESS specification:

```
*)
ENTITY person
  SUBTYPE OF (named_owner);
END_ENTITY;
(*
```

group

A type of car owner consisting of a group of people.

EXPRESS specification:

```
*)
ENTITY group
  SUBTYPE OF (owner);
  members : SET [1:?] OF person;
END_ENTITY;
(*
```

Attribute definitions

members:

The people who form the `tt "group"`.

Function and procedure definitions

dealer_for_mnfs

This function calculates the total number of distinct manufacturers of cars owned by a `tt "garage"`.

Parameters

dealer:

An instance of a `tt "garage"`.

RESULT:

The number of distinct manufacturers of the cars owned by the `tt "garage"`.

EXPRESS specification:

```
*)
FUNCTION dealer_for_mnfs(dealer : garage) : INTEGER;
  LOCAL
    cars : SET OF car := [];
    transfers : SET OF transfer := [];
    makers : SET OF manufacturer := [];
  END_LOCAL;
  transfers := USEDIN(dealer, 'TRANSFER.NEW');
  REPEAT i := 1 TO SIZEOF(transfers);
    cars := cars + transfers[i].item;
  END_REPEAT;
  transfers := USEDIN(dealer, 'TRANSFER.PRIOR');
  REPEAT i := 1 TO SIZEOF(transfers);
    cars := cars - transfers[i].item;
  END_REPEAT;
  REPEAT i := 1 TO SIZEOF(cars);
    makers := makers + cars[i].model_type.made_by;
  END_REPEAT;
  RETURN (SIZEOF(makers));
END_FUNCTION;
(*
```

mnfg_average_consumption

This function calculates the average fuel consumption in a given year of all the cars made by a particular manufacturer.

Parameters

mnfg:

A `tt "manufacturer"`.

when:

An INTEGER representing a particular year.

RESULT:

A REAL giving the average fuel consumption of the manufacturer's cars during a particular year.

EXPRESS specification:

```
*)
FUNCTION mnfg_average_consumption(mnfg : manufacturer;
                                  when : INTEGER) : REAL;
  (* returns the average fuel consumption of the given
     manufacturer's cars produced in the given year *)
  LOCAL
    models : SET OF car_model := [];
    cars   : SET OF car := [];
    num    : INTEGER := 0;
    tot    : INTEGER := 0;
    fuel   : REAL := 0;
    result : REAL := 0.0;
  END_LOCAL;
     -- set of mnfg's models
  models := USEDIN(mnfg, 'MODEL.MADE_BY');
  REPEAT i := 1 TO SIZEOF(models);
     -- cars of particular model year
    cars := QUERY(temp <* USEDIN(models[i], 'CAR.MODEL_TYPE')
            | temp.production_year = when);
    num := SIZEOF(cars);
    fuel := fuel + num*models[i].consumption;
    tot := tot + num;
  END_REPEAT;
  IF tot > 0.0 THEN
    result := fuel/tot;
  END_IF;
  RETURN (result);
END_FUNCTION;
(*
```

Entity classification structure

The following indented listing shows the entity classification structure. Entities in upper case characters are defined in this schema. Entities in lower case characters are defined in other schemas.

```
CAR
    DESTROYED_CAR
CAR_MODEL
OWNER
    GROUP
    NAMED_OWNER
        GARAGE
```

```
        MANUFACTURER
        PERSON
TRANSFER
```

```
*)
END_SCHEMA;  -- end of support schema
(*
```

Calendar schema

This schema contains definitions related to dates and other calendrical items.

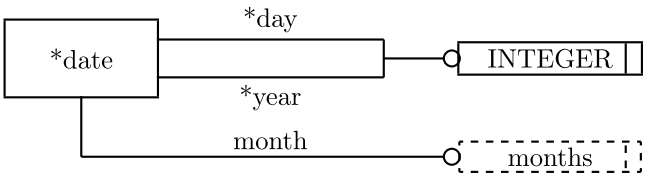


Figure 5. Complete entity-level model of Calendar schema.

Figure Complete entity-level model of Calendar schema. is an EXPRESS-G model showing the contents of this schema.

EXPRESS specification:

```
*)
SCHEMA calendar;
(*
```

Type definitions

months

An enumeration of the months of the year. `tt "January"` is the first month in a year and `tt "December"` is the last month in a year.

EXPRESS specification:

```
*)
TYPE months = ENUMERATION OF
    (January, February, March,
     April,   May,      June,
     July,    August,   September,
     October, November, December);
END_TYPE;
(*
```

Entity definitions

date

A `tt "date"` AD in the Gregorian calendar.

EXPRESS specification:

```
*)
ENTITY date;
  day   : INTEGER;
  month : months;
  year  : INTEGER;
WHERE
  days_ok : {1 <= day <= 31};
  year_ok : year > 0;
  date_ok : valid_date(SELF);
END_ENTITY;
(*
```

Attribute definitions

day:

The day of the `tt "month"`.

month:

The month of the `tt "year"`

year:

The year.

Propositions

days_ok:

The `tt "day"` shall be numbered between 1 and 31 inclusive.

year_ok:

The year shall be greater than zero.

date_ok:

The combination of `tt "day"`, `tt "month"` and `tt "year"` shall form a valid date, taking into account the differing numbers of days in particular months, and also the effect of leap years.

Function and procedure definitions

valid_date

This function checks a `tt "date"` for valid day, month, year combinations.

Parameters

par:

A `tt "date"`.

RESULT:

A Boolean. TRUE if the `tt "date"` has a valid day, month, year combination, FALSE otherwise.

EXPRESS specification:

```
*)
FUNCTION valid_date (par : date) : BOOLEAN;
  (* returns FALSE if its input is not a valid date *)
  CASE par.month OF
    April     : RETURN (par.day <= 30);
    June      : RETURN (par.day <= 30);
    September : RETURN (par.day <= 30);
    November  : RETURN (par.day <= 30);
    February  : IF (leap_year(par.year)) THEN
                  RETURN (par.day <= 29);
                ELSE
                  RETURN (par.day <= 28);
                END_IF;
    OTHERWISE : RETURN (TRUE);
  END_CASE;
END_FUNCTION;
(*
```

leap_year

This function checks whether a given integer could represent a leap year.

Parameters

year:

An INTEGER.

RESULT:

A Boolean. TRUE if `tt "year"` is a leap year, otherwise FALSE.

EXPRESS specification:

```
*)
FUNCTION leap_year(year : INTEGER) : BOOLEAN;
  (* returns TRUE if its input is a leap year *)
```

```
  IF ((((year MOD 4) = 0) AND ((year MOD 100) <> 0)) OR
      ((year MOD 400) = 0)) THEN
    RETURN (TRUE);
  ELSE
    RETURN (FALSE);
  END_IF;
END_FUNCTION;
(*
```

current_date

This function returns the current date.

Parameters

RESULT:

The current `tt "date"`.

EXPRESS specification:

```
*)
FUNCTION current_date : date;
  (* This function returns the date when it is called.
     Typically, it will be implemented via a system provided
     procedure within the information base *)
END_FUNCTION;
(*
```

days_between

This function returns the number of days between any two `tt "date"`s.

Parameters

d1:

A `tt "date"`.

d2:

A `tt "date"`.

RESULT:

An Integer. The number of days between the two input `tt "dates"`. If `tt "d1"` is earlier than `tt "d2"` a positive integer is returned; if `tt "d1"` is later than `tt "d2"` a negative integer is returned; otherwise zero is returned.

EXPRESS specification:

```
*)
```

```
FUNCTION days_between(d1, d2 : date) : INTEGER;
  (* returns the number of days between two input dates. If d1
     is earlier than d2, a positive number is returned. *)
END_FUNCTION;
(*
```

Entity classification structure

The following indented listing shows the entity classification structure. Entities in upper case characters are defined in this schema. Entities in lower case characters are defined in other schemas.

```
DATE
```

```
*)
END_SCHEMA; -- end of calendar schema
(*
```